

# Copione per la presentazione d'esame

La presentazione contiene 20 slide, cinque per ciascun relatore. Durata consigliata: 18-22 minuti.

## Chiara — slide 1-5

### Slide 1 — Titolo

«Il progetto trasforma dati stradali reali di Dublino in un problema PDDL+, lo risolve con ENHSP e mostra il risultato tramite una webapp e SUMO. La presentazione distingue sempre il modello di planning dalla simulazione.»

### Slide 2 — Obiettivo

«L'input è un file OpenStreetMap. Il modello deve rispettare collegamenti diretti, sensi unici, distanze, velocità e semafori. L'output è un piano temporizzato. Non stiamo realizzando un navigatore commerciale: usiamo un modello sperimentale con assunzioni dichiarate.»

### Slide 3 — Architettura

«OSM fornisce i dati; Python costruisce il grafo; PDDL+ descrive il problema; ENHSP trova il piano; la webapp lo presenta. SUMO usa una propria rete e serve per simulare. Nella modalità dinamica SUMO ricalcola con Dijkstra una rotta tra gli stessi estremi, quindi non è una traduzione azione per azione del piano ENHSP.»

### Slide 4 — Struttura OSM

«Un node è un punto con ID e coordinate. Una way è una sequenza ordinata di node e rappresenta una strada o parte di essa. I tag descrivono nome, categoria, velocità e direzione. Nell'esempio Greek Street ha otto node, maxspeed 30 e oneway yes.»

### Slide 5 — Perché si vedono più di 14 nodi

«I conteggi appartengono a livelli diversi. Il file piccolo contiene 555 node grezzi; il grafo contratto ha 201 nodi con archi uscenti; il problema PDDL piccolo contiene 14 oggetti scelti manualmente. Nella webapp lo slider indica un massimo tra 10 e 200, non il numero della baseline piccola.»

Passaggio: «Elisa descrive ora come le way OSM diventano archi PDDL e da dove provengono i valori numerici.»

## Elisa — slide 6-10

### Slide 6 — Nodi per strada

«Una strada OSM non ha un numero fisso di node: ne ha almeno due e può averne decine. Nei nostri file il massimo è 16 nella zona piccola, 67 nella media e 106 nella grande. Quei punti descrivono anche le curve. Un arco PDDL collega invece sempre due incroci consecutivi.»

### Slide 7 — Contrazione

«Filtriamo le categorie percorribili in auto. Consideriamo junction le estremità delle way e i node condivisi da almeno due way. I punti intermedi vengono rimossi come stati, ma le loro distanze vengono sommate. In questo modo riduciamo il problema senza perdere la lunghezza del tratto.»

### Slide 8 — Velocità

«La velocità viene letta dal tag maxspeed della way. Il valore, nei dataset, è espresso in chilometri orari e viene convertito in metri al secondo: 30 diventa 8,33 e 50 diventa 13,89. Tutti gli archi contratti derivati da quella way ereditano il valore. Se manca o non è numerico, il codice usa 30 km/h.»

### Slide 9 — Distanze e direzione

«La distanza viene calcolata con Haversine tra ogni coppia di coordinate consecutive e poi sommata. Se oneway è yes creiamo solo l'arco nel verso OSM; altrimenti aggiungiamo anche il verso opposto. Per questo gli archi PDDL sono diretti.»

### Slide 10 — Semafori

«Un semaforo è un node con il tag highway uguale a traffic\_signals. Il parser conserva il suo ID e nel problema generato assegna signal-delay 30 a quel node, zero agli altri. Media e grande usano questa corrispondenza esatta. La baseline piccola è manuale e assegna quattro ritardi anche a incroci vicini ai segnali, a distanze dichiarate tra 14 e 68 metri.»

Passaggio: «Definiti i dati dell'istanza, Emanuele spiega perché il dominio è modellato con azioni, processi ed eventi.»

## Emanuele — slide 11-15

### Slide 11 — Domain e problem

«Il domain contiene le regole generali ed è riutilizzato. I problem contengono le mappe specifiche. Questa separazione permette di cambiare città o sottografo senza riscrivere il comportamento del veicolo.»

### Slide 12 — Problema

«Gli oggetti sono gli incroci. Road indica un arco diretto. Distance e speed sono costanti dell'arco; progress è lo stato dinamico; signal-delay è il costo del nodo di arrivo. At definisce partenza e obiettivo. La metrica dichiarata è total-time.»

### Slide 13 — Stato del dominio

«At, road e moving sono predicati booleani. Distance, speed e signal-delay arrivano dal problema. Progress cambia continuamente. Total-dist e total-time accumulano il costo del percorso completato.»

### Slide 14 — Azione, processo, evento

«Start-move è la decisione del planner. Driving è un processo continuo che aumenta progress con speed per il tempo trascorso. Arrive è un evento automatico quando progress raggiunge distance. PDDL+ è necessario perché PDDL classico non rappresenta direttamente processi continui ed eventi automatici.»

### Slide 15 — Tempo e semafori

«All'arrivo aggiungiamo distance diviso speed e signal-delay. Total-time non cresce continuamente: viene aggiornato negli eventi per ridurre la complessità del planner. È importante dichiarare un limite: i 30 secondi penalizzano numericamente il piano, ma non impongono uno stop di 30 secondi nella sequenza temporale delle azioni.»

Passaggio: «Pierluigi conclude con l'interfaccia, i risultati verificati e il ruolo reale di SUMO.»

## Pierluigi — slide 16-20

### Slide 16 — Webapp

«La prima chiamata carica il file, costruisce il grafo e restituisce al massimo N nodi. L'utente può scegliere start e goal; il backend verifica che il goal sia raggiungibile rispettando i sensi unici. Solve genera il PDDL e avvia ENHSP. Il tempo di planning è ora mostrato correttamente in millisecondi.»

### Slide 17 — Risultati

«I risultati sono stati ricalcolati dai file e dai piani il 7 giugno 2026. Piccola: 1570 metri, quattro segnali sul piano e costo 308,5 secondi. Media: 1623 metri, due dei sei segnali selezionati sul piano e costo 210,5 secondi. Grande: 1335 metri, nessun segnale sul piano e costo 141,6 secondi. Il planning ha richiesto rispettivamente circa 100, 141 e 80 millisecondi in questa esecuzione.»

### Slide 18 — SUMO

«SUMO usa junction ed edge del net.xml. Gli ID OSM permettono di individuare gli estremi e Dijkstra trova una sequenza di edge. La modalità dinamica usa start e goal del PDDL, non l'intera sequenza ENHSP. SUMO aggiunge poi accelerazione, frenata e semafori propri. È quindi una simulazione complementare, non una prova formale dell'ottimalità PDDL.»

### Slide 19 — Limiti

«Il modello usa un ritardo fisso, non include traffico, riduce la rete e tratta principalmente maxspeed numerici. La piccola è manuale, le altre istanze sono generate. ENHSP aibr trova un piano, ma non presentiamo una dimostrazione di ottimalità globale. Dichiarare questi limiti evita affermazioni non supportate dal codice.»

### Slide 20 — Conclusione

«Il risultato principale è una pipeline riproducibile che collega dati geografici reali, modellazione PDDL+, planning e visualizzazione. Gli sviluppi naturali sono attese semaforiche temporali reali, traffico dinamico, traduzione completa del piano in SUMO e ottimizzazione multi-obiettivo.»

## Risposte brevi alle domande probabili

Perché PDDL+? Perché driving modifica continuamente progress e arrive deve scattare automaticamente.

Una strada quanti nodi ha? Una way OSM ha un numero variabile di node; ogni arco PDDL contratto ha esattamente due estremi.

Da dove viene la velocità? Dal tag maxspeed della way OSM, convertito da km/h a m/s; fallback 30 km/h.

Come individuate i semafori? Dal tag del node `<tag k="highway" v="traffic_signals"/>`.

Il percorso è sicuramente ottimo? Il problema dichiara la minimizzazione di total-time ed ENHSP aibr trova un piano, ma il progetto non produce una prova indipendente di ottimalità globale.

SUMO esegue esattamente il piano? Non nella modalità dinamica attuale: ricostruisce una rotta sulla rete SUMO tra gli stessi start e goal.